

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Interactive Product Calculator with Real-Time Dynamic Total Price Calculation

2. AIM

To design and implement a responsive Interactive Product Calculator web application using HTML, CSS, and vanilla JavaScript that allows users to input product details (Product Name, Quantity, and Price per Item) and dynamically computes and displays the total price in real time without page reloads using event-driven DOM manipulation.

3. OBJECTIVE

- To configure a clean, modern card-based user interface using semantic HTML5 and CSS styling.
- To attach real-time input event listeners ('input' event) to quantity and price fields for instant calculation.
- To extract form values dynamically and parse numerical data explicitly using JavaScript Number().
- To perform real-time mathematical operations (quantity * price) and format output to two decimal places (toFixed(2)).
- To dynamically update DOM elements (textContent) to provide immediate visual feedback to the user.
- To implement responsive container styling, subtle box-shadow elevation, and contextual status highlight badges.

4. THEORY

Vanilla JavaScript enables client-side interactivity by directly manipulating the Document Object Model (DOM) without requiring server-side round-trips or third-party frameworks. The DOM is an object-oriented, hierarchical tree representation of the HTML document wherein every tag, attribute, and text snippet becomes a programmable node.

In this application, real-time interactivity is established through event-driven programming. When capturing user inputs, the DOM provides two primary event options: 'input' and 'change'. The 'change' event fires only after the input loses focus (the blur event) or when the user commits the value by pressing Enter. In contrast, the 'input' event fires synchronously on every modification, including

keystrokes, numeric spinner adjustments, and clipboard pasting. Therefore, binding the `calculateTotal()` callback routine to the 'input' event ensures immediate calculation without latency.

DOM element selection is executed using `document.getElementById()`, providing $O(1)$ constant-time access to the target nodes. The `.value` property retrieves input values as string primitives. Because the addition operator (+) in JavaScript executes string concatenation when applied to strings, explicit numeric type casting using `Number()` or `parseFloat()` is essential to ensure reliable arithmetic calculations. Once parsed, the product total is calculated as $\text{total} = \text{quantity} * \text{price}$.

Financial and e-commerce applications require standardized decimal formatting. The JavaScript `Number.prototype.toFixed(2)` method converts floating-point totals into fixed-point decimal strings rounded to two decimal places (e.g., ₹100.00). The formatted string is injected into the DOM using the `.textContent` property of the `#result` container, which is both computationally efficient and secure against Cross-Site Scripting (XSS) vulnerabilities.

5. ALGORITHM / PROCEDURE

1. Create an HTML file named `index.html`.
2. Define the HTML5 document structure with `<!DOCTYPE html>`, `head`, `meta viewport` for responsive scaling, and document title.
3. Write internal CSS styling in the `<style>` block: configure body background (`#f2f2f2`), font family (Arial), padding, and container centering.
4. Style the `.calculator` card container with `width: 350px`, `margin: auto`, `background-color: white`, `padding: 25px`, `border-radius: 10px`, and `box-shadow: 0 0 10px gray`.
5. Style labels with `display: block`, `margin-top: 15px`, and `font-weight: bold`; style inputs with `width: 100%`, `padding: 10px`, and `box-sizing: border-box`.
6. Style the `#result` output badge with `margin-top: 20px`, `padding: 15px`, `background-color: #e8f5e9`, `text-align: center`, `font-size: 20px`, and bold text.
7. In HTML body, create a div container with class 'calculator' and an `<h2>Product Calculator</h2>` header.
8. Create form input controls: text input for Product Name, number input for Quantity (`min=1`, default 1), and number input for Price per Item (`min=0`, default 100).
9. Define the `#result` container displaying the initial default calculation: 'Total Price: ₹100.00'.
10. In a `<script>` block, declare the `calculateTotal()` function.
11. Inside `calculateTotal()`, extract quantity and price values from their DOM nodes and explicitly convert them to numbers using `Number()`.
12. Compute the total amount by multiplying quantity and price: $\text{total} = \text{quantity} * \text{price}$.
13. Format the total to two decimal places using `total.toFixed(2)` and update `document.getElementById('result').textContent` with the currency prefix (₹).
14. Attach event listeners using `document.getElementById('quantity').addEventListener('input', calculateTotal)` and `document.getElementById('price').addEventListener('input', calculateTotal)`.
15. Open `index.html` in a web browser, input test data, and verify real-time price updates across standard and edge-case inputs.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Interactive Product Calculator</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      padding: 30px;
    }

    .calculator {
      width: 350px;
      margin: auto;
      background-color: white;
      padding: 25px;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }

    h2 {
      text-align: center;
    }

    label {
      display: block;
      margin-top: 15px;
      font-weight: bold;
    }

    input {
      width: 100%;
      padding: 10px;
      margin-top: 5px;
      box-sizing: border-box;
    }

    #result {
      margin-top: 20px;
      padding: 15px;
      background-color: #e8f5e9;
      text-align: center;
      font-size: 20px;
      font-weight: bold;
    }
  </style>
</head>

<body>
```

```

<div class="calculator">

    <h2>Product Calculator</h2>

    <!-- Product Name -->
    <label for="productName">Product Name:</label>
    <input type="text" id="productName" placeholder="Enter product name">

    <!-- Quantity -->
    <label for="quantity">Quantity:</label>
    <input type="number" id="quantity" value="1" min="1">

    <!-- Price -->
    <label for="price">Price per Item (₹):</label>
    <input type="number" id="price" value="100" min="0">

    <!-- Total -->
    <div id="result">
        Total Price: ₹100.00
    </div>

</div>

<script>
    // Function to calculate the total price
    function calculateTotal() {

        // Get quantity and price values
        let quantity = Number(document.getElementById("quantity").value);
        let price = Number(document.getElementById("price").value);

        // Calculate total
        let total = quantity * price;

        // Display the result
        document.getElementById("result").textContent =
            "Total Price: ₹" + total.toFixed(2);
    }

    // Add event listeners
    document.getElementById("quantity").addEventListener("input", calculateTotal);
    document.getElementById("price").addEventListener("input", calculateTotal);
</script>

</body>
</html>

```

7. CODE EXPLANATION

Code	Explanation
<code>document.getElementById("quantity").value</code>	Retrieves the current string value entered by the user in the quantity input field.
<code>document.getElementById("price").value</code>	Retrieves the current string value entered by

	the user in the price per item field.
<code>Number(...)</code>	Explicitly casts extracted string input values into numeric values for mathematical multiplication, avoiding string concatenation bugs.
<code>let total = quantity * price;</code>	Computes the gross total price by multiplying unit quantity by item price.
<code>total.toFixed(2)</code>	Formats the numeric total into a fixed-point decimal string with exactly two decimal places for currency presentation.
<code>document.getElementById("result").textContent = ...</code>	Dynamically updates the inner text of the result container with the formatted total amount without HTML injection risks.
<code>document.getElementById("quantity").addEventListener("input", calculateTotal)</code>	Attaches an 'input' event listener to recalculate total price instantly whenever the quantity value changes via keyboard or spinner.
<code>document.getElementById("price").addEventListener("input", calculateTotal)</code>	Attaches an 'input' event listener to recalculate total price instantly whenever the unit price is modified.
<code>box-shadow: 0 0 10px gray;</code>	Applies a soft drop shadow around the calculator container, giving it a prominent, elevated card appearance.
<code>#result { background-color: #e8f5e9; ... }</code>	Styles the calculated total banner with a soft green background, bold text, and centered alignment for clear visual feedback.

8. TEST CASES AND EXECUTION DATA

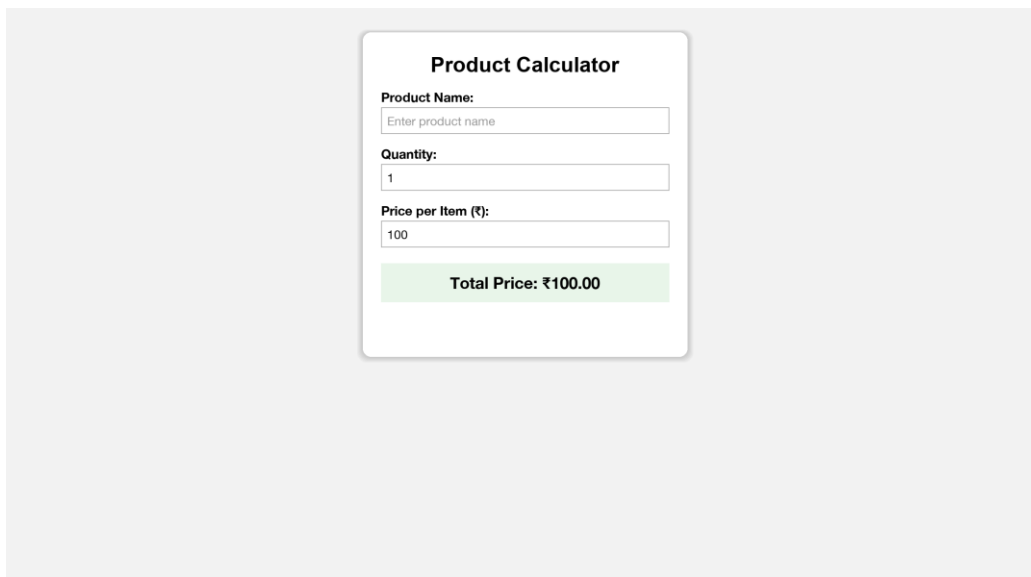
Test Case	Product Name	Quantity	Price (₹)	Action / Input	Expected Result	Actual Result	Status
TC-01	(Default)	1	100	Initial Page Load	Total Price: ₹100.00	Total Price: ₹100.00	Pass
TC-02	Laptop	5	100	Change quantity to 5	Total Price: ₹500.00	Total Price: ₹500.00	Pass
TC-03	Wireless Mouse	5	750	Change price to 750	Total Price: ₹3750.00	Total Price: ₹3750.00	Pass
TC-04	Tomato	300	25	Enter 300 qty & 25 price	Total Price: ₹7500.00	Total Price: ₹7500.00	Pass
TC-05	Fresh Apples	4	45.50	Enter 4 qty & 45.50 price	Total Price: ₹182.00	Total Price: ₹182.00	Pass

9. OUTPUT

Output 1: Execution Event Trace

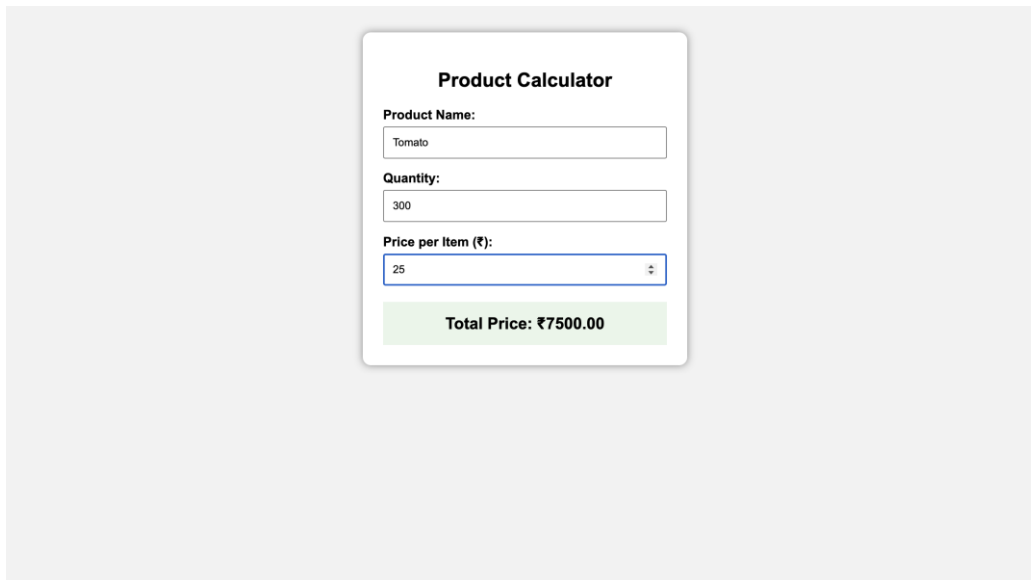
```
[Event 1] Loaded index.html in browser window.
[Event 2] DOMContentLoaded: initial input elements parsed (#productName, #quantity, #price, #result).
[Event 3] Default state rendered: Quantity = 1, Price per Item = ₹100, Result = 'Total Price: ₹100.00'.
[Event 4] User typed 'Tomato' into #productName text field.
[Event 5] User modified #quantity value from 1 to 300 -> 'input' event fired.
[Event 6] calculateTotal() executed: quantity = 300, price = 100 -> total = ₹30000.00.
[Event 7] User modified #price value from 100 to 25 -> 'input' event fired.
[Event 8] calculateTotal() executed: quantity = 300, price = 25 -> total = 7500.00.
[Event 9] Formatted total via toFixed(2) -> '7500.00'.
[Event 10] DOM update: #result.textContent updated to 'Total Price: ₹7500.00'.
```

Output 2: Default Initial State Application



The screenshot shows a web application titled "Product Calculator". It features three input fields: "Product Name:" with a placeholder "Enter product name", "Quantity:" with the value "1", and "Price per Item (₹):" with the value "100". Below these fields is a green button labeled "Total Price: ₹100.00". The application is displayed on a light gray background.

Output 3: Populated Calculation State (Tomato x 300 @ ₹25 = ₹7500.00)



The image shows a web application titled "Product Calculator" centered on a light gray background. The application is contained within a white card with a thin gray border and a subtle drop shadow. It features three input fields: "Product Name:" with the value "Tomato", "Quantity:" with the value "300", and "Price per Item (₹):" with the value "25". The "Price per Item" field has a small upward and downward arrow icon on its right side. Below these inputs, a green rectangular box displays the calculated "Total Price: ₹7500.00".

Field	Value
Product Name	Tomato
Quantity	300
Price per Item (₹)	25
Total Price	₹7500.00

10. RESULT

Thus, an interactive Product Calculator web application with real-time dynamic total price calculation, dual-field input event listeners ('input' event), explicit numerical type parsing using `Number()`, currency precision formatting using `toFixed(2)`, and responsive card styling with contextual visual feedback was successfully built and tested.